

Expansion Statements

Document #: P1306R5 [\[Latest\]](#) [\[Status\]](#)
Date: 2025-06-20
Project: Programming Language C++
Audience: CWG
Reply-to: Dan Katz
<dkatz85@bloomberg.net>
Andrew Sutton
<andrew.sutton@beyondidentity.com>
Sam Goodrick
<samuel.goodrick@beyondidentity.com>
Daveed Vandevoorde
<daveed@edg.com>
Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[3 Design](#)

[3.1 Expansion over Expression Lists](#)

[3.2 Expansion over Ranges](#)

[3.3 Expansion over Tuples](#)

[3.4 Prioritizing Range over Tuple](#)

[3.5 `break` and `continue`](#)

[3.6 Expansion over Types](#)

[3.7 Implementation experience](#)

[4 Proposed wording](#)

[4.1 Feature-test-macro](#)

[5 References](#)

§ 1 Revision History

[\[P1306R4\]](#) and R5: Rewrote the prose and the wording.

[\[P1306R3\]](#) Expansion over a range requires a constant expression. Added support for `break` and `continue` control flow during evaluation.

[\[P1306R2\]](#) Adoption of `template for` syntax. Added support for init-statement, folded pack expansion into new expansion-init-list mechanism. Updated reflection code to match P2996. Minor updates to wording: updated handling of switch statements, work around lack of general non-transient `constexpr` alloca-

tion, eliminated need for definition of an “intervening statement”, rebased onto working draft, updated feature macro value, fixed typos. Addressed CWG review feedback

[P1306R1] Adopted a unified syntax for different forms of expansion statements. Further refinement of semantics to ensure expansion can be supported for all traversable sequences, including ranges of input iterators. Added discussion about `break` and `continue` within expansions.

[P1306R0] superceded and extended [P0589R0] (Tuple-based for loops) to work with more destructurable objects (e.g., classes, parameter packs). Added a separate `constexpr-for` variant that a) makes the loop variable a constant expression in each repeated expansion, and b) makes it possible to expand `constexpr` ranges. The latter feature is particularly important for static reflection.

§ 2 Introduction

This paper proposes a new kind of statement that enables the compile-time repetition of a statement for each element of a tuple, array, class, range, or brace-delimited list of expressions. Existing methods for iterating over a heterogeneous container inevitably leverage recursively instantiated templates to allow some part of the repeated statement to vary (e.g., by type or constant) in each instantiation. While such behavior can be encapsulated in a single library operation (e.g., Boost.Hana’s `for_each`) or, potentially in the future, using the `[:expand(...):]` construct built on top of [P2996R10] (Reflection for C++26) reflection facilities, there are several reasons to prefer language support:

First, repetition is a fundamental building block of algorithms, and should be expressible directly without complex template instantiation strategies.

Second, such repetition should be as inexpensive as possible. Recursively instantiating templates generates a large number of specializations, which can consume significant compilation time and memory resources.

Third, library-based approaches rely on placing the repeated statements in a lambda body, which changes the semantics of something like a `return` statement — and makes coroutines unusable.

Lastly, “iteration” over destructuring classes effectively requires language support to implement correctly.

Here are some basic usage examples:

Today	Proposed
<pre>void print_all(std::tuple<int, char> xs) { hana::for_each(xs, [&](auto elem){ std::println("{} ", elem); }); }</pre>	<pre>void print_all(std::tuple<int, char> xs) { template for (auto elem : xs) { std::println("{} ", elem); } }</pre>
<pre>template <class... Ts> void print_all(Ts... xs) {</pre>	<pre>template <class... Ts> void print_all(Ts... xs) { template for (auto elem : {xs...}) {</pre>

Today	Proposed
<pre>hana::for_each(std::tie(xs...), [&] (auto elem){ std::println("{} ", elem); }); }</pre>	<pre>std::println("{} ", elem); } }</pre>
<pre>template <class T> void print_all(T const& v) { [: expand(nsdms(^^T)) :] >> [&]<auto e>{ std::println(".{}={}", identifier_of(e), v.[:e:]); }; }</pre>	<pre>template <class T> void print_all(T const& v) { template for (constexpr auto e : define_static_array(nsdms(^^T))) { std::println(".{}={}", identifier_of(e), v.[:e:]); } }</pre>

For the last row, `expand` is demonstrated in [\[P2996R10\]](#), `define_static_array()` comes from [\[P3491R1\]](#) (`define_static_{string,object,array}`). (although can be implemented purely on top of p2996) and works around non-transient allocation (more on this later), and `nsdms(type)` is just shorthand for `nonstatic_data_members_of(type, std::meta::access::unprivileged())` just to help fit.

§ 3 Design

The proposed design allows iterating over:

- expression lists,
- anything destructurable via structured bindings (i.e. tuple-like), or
- ranges with compile-time size

The expansion statement

```
template for (init-statementopt for-range-declaration : expansion-initializer) compound-statement
```

will determine an *expansion size* based on the *expansion-initializer* and then *expand* into:

```
{
    init-statementopt
    additional-expansion-declarationsopt; // depends on expansion kind

    {
        for-range-declaration = E(0);
        compound-statement
    }
}
```

```

{
    for-range-declaration = E(1);
    compound-statement
}

// ... repeated up to ...

{
    for-range-declaration = E(expansion-size - 1);
    compound-statement
}

}

```

The mechanism of determining the *additional-expansion-declarations* (if any), the expansion size, and *E* depends on the *expansion-initializer*.

§ 3.1 Expansion over Expression Lists

If *expansion-initializer* is of the form `{ expression-list }`, then:

- there are no *additional-expansion-declarations*
- the expansion size is the number of *expressions* in the *expression-list* (possibly 0), and
- *get-expr(i)* is the *i*th *expression* in the *expression-list*.

For example:

Code	Expands Into
<pre> template <typename... Ts> void print_all(Ts... elems) { template for (auto elem : {elems...}) { std::println("{} ", elem); } } </pre>	<pre> template <typename... Ts> void print_all(Ts... elems) { { auto elem = elems...[0]; std::println("{} ", elem); } { auto elem = elems...[1]; std::println("{} ", elem); } { auto elem = elems...[2]; std::println("{} ", elem); } } </pre>

Approximately anyway. The *expression-list* need not be a simple pack expansion for which pack indexing applies, that's just for illustration purposes.

An earlier revision of this paper did not have dedicated syntax for expansion over packs. The syntax for the above example was originally proposed as:

```
template <typename... Ts>
void print_all(Ts... elems) {
    template for (auto elem : elems) { // just elems
        std::println("{} ", elem);
    }
}
```

This was pointed out by Richard Smith to be ambiguous [on the EWG reflector](#). Consider:

```
template <typename... Ts>
void fn(Ts... vs) {
    ([&](auto p){
        template for (auto& v : vs) {
            // ...
        }
    })(vs), ...);
}
```

Consider the call `fn(array{1, 2, 3, 4}, array{1, 3, 5, 7}, array{2, 4, 6, 8})`. It is far from clear whether the expansion statement containing `vs` expands over:

- each of the three array arguments (once for each invocation of the lambda), or
- each of the four `int` elements (of a different array for each invocation of the lambda).

Initially, support for pack iteration was dropped from the proposal entirely, but it was added back using the *expansion-init-list* syntax in [\[P1306R2\]](#).

In addition to avoiding ambiguity, it is also broadly more useful than simply expanding over a pack since it allows ad hoc expressions. For instance, can add prefixes, suffixes, or even multiple packs: `{0, xs..., 1, ys..., 2}` is totally fine.

§ 3.2 Expansion over Ranges

If *expansion-initializer* is a single expression that is a range, then:

- *addition-expansion-declarations* is:

```
constexpropt auto&& __range = expansion-initializer;
constexpropt auto __begin = begin-expr; // see [stmt.ranged]
```

where the `constexpr` specifier is present when the *for-range-declaration* is declared with `constexpr`.

- the expansion size is $end\text{-}expr - __begin$. This expression must be a constant expression. It is possible for this to be the case even if `__begin` is not `constexpr`, but expansion statements over ranges in general are really only useful if the loop element is `constexpr`.
- `get-expr(i)` is `*(__begin + i)`.

For example:

Code	Expands Into
<pre>void f() { template for (constexpr int I : std::array{1, 2, 3}) { static_assert(I < 4); } }</pre>	<pre>void f() { { constexpr auto&& __range = std::array{1, 2, 3}; constexpr auto __begin = __range.begin(); constexpr auto __expansion-size = __range.end() - __begin; // 3 { constexpr int I = *(__begin + 0); static_assert(I < 4); } { constexpr int I = *(__begin + 1); static_assert(I < 4); } { constexpr int I = *(__begin + 2); static_assert(I < 4); } } }</pre>

Note that the `__range` variable is declared `constexpr` here. As such, all the usual rules for `constexpr` variables apply. Including the restriction on non-transient allocation.

Consider:

```
template <typename T>  
void print_members(T const& v) {  
    template for (constexpr auto r : nonstatic_data_members_of(^T)) {  
        std::println(".{}={}", identifier_of(r), v.[:r:]);  
    }  
}
```

Examples like this feature prominently in [\[P2996R10\]](#). And at first glance, this seems fine. The compiler knows the length of the vector returned by `members_of(^T)`, and can expand the body for each element. However, the expansion in question more or less requires a `constexpr` vector, which the language is not yet equipped to handle.

We at first attempted to carve out a narrow exception from `[expr.const]` to permit non-transient `constexpr` allocation in this very limited circumstance. Although the wording seemed reasonable, our implementation experience with Clang left us less than optimistic for this approach: The architecture of Clang’s constant evaluator really does make every effort to prevent dynamic allocations from surviving the evaluation of a constant expression (certainly necessary to produce a “`constexpr` vector”). After some wacky experiments that amounted to trying to “rip the constant evaluator in half” (i.e., separating the “evaluation state”, whereby dynamically allocated values are stored, from the rest of the metadata pertaining to an evaluation), we decided to fold: as of the [\[P1306R3\]](#) revision, we instead propose restricting expansion over expansion-iterable expressions to only cover those that are constant expression.

In other words — the desugaring described above (which is similar to the desugaring for the C++11 range-based `for` statement) — is what you get. No special cases.

Regrettably, this makes directly expanding over `members_of(^T)` ill-formed for C++26 — but all is not lost: By composing `members_of` with the `define_static_array` function from [\[P3491R1\]](#) (`define_static_{string,object,array,}`) we obtain a `constexpr` span containing the same reflections from `members_of`:

```
template <typename T>
void print_members(T const& v) {
    template for (constexpr auto r : define_static_array(nonstatic_
        data_members_of(^T))) {
        std::println(".{}={}", identifier_of(r), v.[:r:]);
    }
}
```

This works fine, since we no longer require non-transient allocation. We’re good to go.

This yields the same expressive power, at the cost of a few extra characters and a bit more memory that must be persisted during compilation. It’s a much better workaround than others we have tried (e.g., the expand template), and if (when?) WG21 figures out how to support non-transient `constexpr` allocation, the original syntax should be able to “just work”.

§ 3.3 Expansion over Tuples

If *expansion-initializer* is a single expression that is a range, then:

- the expansion size is the *structured binding size* of the *expansion-initializer* ([9.7 \[dcl.struct.bind\]](#))
- *addition-expansion-declarations* is:

```
constexpr_opt auto&& [__v0, __v0, ..., __v_expansion_size-1] = expansion_initializer;
```

- `get_expr(i)` is `__vi` if either the referenced type is an lvalue reference or the *expansion-initializer* is an lvalue. Otherwise, `std::move(__vi)`.

For example:

Code	Desugars Into
<pre>auto tup = std::make_tuple(0, 'a'); template for (auto& elem : tup) { elem += 1; }</pre>	<pre>auto tup = std::make_tuple(0, 'a'); { auto&& [__v₀, __v₁] = tup; { auto& elem = __v₀; elem += 1; } { auto& elem = __v₁; elem += 1; } }</pre>

§ 3.4 Prioritizing Range over Tuple

Most types can either be used as a range or destructured, but not both. And even some that can be used in both contexts have equivalent meaning in both — C arrays and `std::array`.

However, it is possible to have types that have different meanings with either interpretation. That means that, for a given type, we have to pick one interpretation. Which should we pick?

One such example is `std::ranges::subrange(first, last)`. This could be:

- as a range, from `[first, last)`.
- as a tuple, specifically the iterators `first` and `last` (i.e. always size 2).

Another such example is a range type that just happens to have all public members. `std::views::empty<T>` isn't going to have any non-static data members at all, so it's tuple-like (with size 0) and also a range (with size 0), so that one amusingly works out the same either way.

But any other range whose members happen to be public probably wants to be interpreted as a range. Moreover, the structured binding rule doesn't actually require *public* members, just *accessible* ones. So there are some types that might be only ranges externally but could be both ranges and tuples internally.

In all of these cases, it seems like the obviously desired interpretation is as a range. Which is why we give priority to the range interpretation over the tuple interpretation.

Additionally, given a type that can be interpreted both ways, it's easy enough to force the tuple interpretation if so desired:

```
template <class T>
constexpr auto into_tuple(T const& v) {
    auto [...parts] = v;
    return std::tie(parts...);
}
```

§ 3.5 break and continue

Earlier revisions of the paper did not support `break` or `continue` within expansion statements. There was previously concern that users would expect such statement to exercise control over the code generation / expansion process at translation time, rather than over the evaluation of the statement.

Discussions with others have convinced us that this will not be an issue, and to give the keywords their most obvious meaning: `break` jumps to just after the end of the last expansion, whereas `continue` jumps to the start of the next expansion (if any).

§ 3.6 Expansion over Types

There are regular requests to support expanding over *types* directly, rather than expressions:

```
template <typename... Ts>
void f() {
    // strawman syntax
    template for (typename T : {Ts...}) {
        do_something<T>();
    }
}
```

Something like this would be difficult to support directly since you can't tell that the declaration is just a type rather than an unnamed variable. But with Reflection coming, there's less motivation to come up with a way to address this problem directly since we can just iterate in the value domain:

```
template <typename... Ts>
void f() {
    template for (constexpr auto r : {^Ts...}) {
        using T = [:r:];
        do_something<T>();
    }
}
```

§ 3.7 Implementation experience

Bloomberg's Clang/P2996 fork (available [on Godbolt](#)) implements all features proposed by this paper. Expansion statements are enabled with the `-fexpansion-statements` flag (or with `-freflection-lat-est`).

§ 4 Proposed wording

Update 6.4.2 [\[basic.scope.pdecl\]](#)/11 to specify the locus of an expansion statement:

- 11 The locus of a *for-range-declaration* of a range-based `for` statement ([stmt.range]) is immediately after the *for-range-initializer*. The locus of a *for-range-declaration* of an expansion statement ([stmt.expand]) is immediately after the *expansion-initializer*.

Update 6.4.3 [\[basic.scope.block\]](#)/1.1 to include expansion statements:

- (1.1) — selection, ~~or~~ iteration, or expansion statement ([stmt.select], [stmt.iter], [stmt.expand])

Modify 6.7.7 [\[class.temporary\]](#)/5 to stop counting contexts:

- 5 There are ~~five~~ several contexts in which temporaries are destroyed at a different point than the end of the full-expression. [...]

Insert a new paragraph after 6.7.7 [\[class.temporary\]](#)/7 to extend the lifetime of temporaries created by expansion statements, and update the ordinal number used in paragraph 8:

- 7 The fourth context is when a temporary object is created in the *for-range-initializer* of either a range-based `for` statement or an enumerating expansion statement ([stmt.expand]). If such a temporary object would otherwise be destroyed at the end of the *for-range-initializer full-expression*, the object persists for the lifetime of the reference initialized by the *for-range-initializer*.
- 7+ The fifth context is when a temporary object is created in the *expansion-initializer* of an iterating or destructuring expansion statement. If such a temporary object would otherwise be destroyed at the end of that *expansion-initializer*, the object persists for the lifetime of the reference initialized by the *expansion-initializer*, if any.
- 8 The ~~fifth~~ sixth context is when a temporary object is created in a structured binding declaration ([decl.struct.bind]). [...]

Add a production for expansion statements to *statement* to 8.1 [\[stmt.pre\]](#). Also move the grammar for *for-range-declaration* from [stmt.iter.general] to here:

- 1 Except as indicated, statements are executed in sequence.

```

statement:
    labeled-statement
    attribute-specifier-seqopt expression-statement
    attribute-specifier-seqopt compound-statement
    attribute-specifier-seqopt selection-statement
    attribute-specifier-seqopt iteration-statement
+   attribute-specifier-seqopt expansion-statement
    attribute-specifier-seqopt jump-statement
    declaration-statement
    attribute-specifier-seqopt try-block

init-statement:
    expression-statement
    simple-declaration
    alias-declaration

condition:
    expression
    $attribute-specifier-seqopt decl-specifier-seq declarator brace-
    or-equal-initializer
    structured-binding-declaration initializer

+ for-range-declaration:
+   attribute-specifier-seqopt decl-specifier-seq declarator
+   structured-binding-declaration
+
+ for-range-initializer:
+   expr-or-braced-init-list

See [dcl.meaning] for the optional attribute-specifier-seq in a for-range-
declaration.

```

Extend “substatement” to cover expansion statements in 8.1 [stmt.pre]/2:

- 2 A *substatement* of a *statement* is one of the following:
- (2.1) — for a *labeled-statement*, its *statement*,
 - (2.2) — for a *compound-statement*, any *statement* of its *statement-seq*,
 - (2.3) — for a *selection-statement*, any of its *statements* or *compound-statements* (but not its *init-statement*), ~~or~~
 - (2.4) — for an *iteration-statement*, its *statement* (but not an *init-statement*); ~~or~~
 - (2.5) — for an *expansion-statement*, its *compound-statement* (but not an *init-statement*).

Extend “enclose” to cover expansion statements in 8.1 [stmt.pre]/3:

3 A statement S1 encloses a statement S2 if

- (3.1) — S2 is a substatement of S1,
- (3.2) — S1 is a *selection-statement*, *or iteration-statement*, *or expansion-statement*, and S2 is the *init-statement* of S1,
- (3.3) — [...]

Extend 8.1 [\[stmt.pre\]](#)/8 to cover *for-range-declarations*:

- 8 In the *decl-specifier-seq* of a condition *or of a for-range-declaration*, including that of any *structured-binding-declaration* of the condition, each *decl-specifier* shall either be a *type-specifier* or *constexpr*. The *decl-specifier-seq* of a *for-range-declaration* shall not define a class or enumeration.

Add a new paragraph to the end of 8.2 [\[stmt.label\]](#):

- 4 An identifier label shall not be enclosed by an *expansion-statement* ([\[stmt.expand\]](#)).

Strike the productions for *for-range-declaration* and *for-range-initializer* from [\[stmt.iter.general\]](#), as they've been moved to [\[stmt.pre\]](#):

```
iteration-statement:
    while ( condition ) statement
    do statement while ( expression ) ;
    for ( init-statement conditionopt ; expressionopt ) statement
    for ( init-statementopt for-range-declaration : for-range-initial-
        izer ) statement
```

- *for-range-declaration*:
- *attribute-specifier-seq_{opt} decl-specifier-seq declarator*
- *structured-binding-declaration*
-
- *for-range-initializer*:
- *expr-or-braced-init-list*

~~See [\[del.meaning\]](#) for the optional *attribute-specifier-seq* in a *for-range-declaration*.~~

Strike 8.6.5 [\[stmt.ranged\]](#)/2, as it's been integrated into [\[stmt.pre\]](#)/8.

- 2 ~~In the *decl-specifier-seq* of a *for-range-declaration*, each *decl-specifier* shall be either a *type-specifier* or *constexpr*. The *decl-specifier-seq* shall not define a~~

Insert this section after 8.6 [\[stmt.iter\]](#) (and renumber accordingly).

Expansion statements [stmt.expand]

- 1 Expansion statements specify repeated instantiations ([temp.decls.general]) of their substatement.

```
expansion-statement:
    template for ( init-statementopt for-range-declaration : expansion-
        initializer ) compound-statement
```

```
expansion-initializer:
    expression
    expansion-init-list
```

```
expansion-init-list:
    { expression-listopt }
```

- 2 The *compound-statement* of an *expansion-statement* is a control-flow-limited statement ([stmt.label]).

- 3 For an expression *E*, let the expressions *begin-expr* and *end-expr* be determined as specified in [stmt.ranged]. An expression is *expansion-iterable* if it does not have array type and either

- (3.1) — *begin-expr* and *end-expr* are of the form *E*.*begin*() and *E*.*end*() or
- (3.2) — argument-dependent lookups for *begin*(*E*) and for *end*(*E*) each find at least one function or function template.

- 4 An expansion statement is

- (4.1) — an *enumerating expansion statement* if its *expansion-initializer* is of the form *expansion-init-list*;
- (4.2) — otherwise, an *iterating expansion statement* if its *expansion-initializer* is an *expansion-iterable* expression;
- (4.3) — otherwise, a *destructuring expansion statement*.

- 5 An expansion statement *S* is equivalent to a *compound-statement* containing instantiations of the *for-range-declaration* (including its implied initialization), together with the *compound-statement* of *S*, as follows:

- (5.1) — If *S* is an enumerating expansion statement, *S* is equivalent to:

```
{
    init-statement
    S0
```

```

...
SN-1
}

```

where N is the number of elements in the *expression-list*, S_i is

```

{
  for-range-declaration = Ei ;
  compound-statement
}

```

and E_i is the i^{th} element of the *expression-list*.

(5.2) — Otherwise, if S is an iterating expansion statement, S is equivalent to:

```

{
  init-statement
  static constexpr auto&& range = expansion-initializer ;
  static constexpr auto begin = begin-expr; // see [stmt.ranged]
  static constexpr auto end = end-expr;    // see [stmt.ranged]

  S0
  ...
  SN-1
}

```

where N is the result of evaluating the expression

```

[] consteval {
  std::ptrdiff_t result = 0;
  for (auto i = begin; i != end; ++i, ++result) ;
  return result; // distance from begin to end
}()

```

and S_i is

```

{
  static constexpr auto iter = begin + i;
  for-range-declaration = *iter;
  compound-statement
}

```

The variables *range*, *begin*, *end*, and *iter* are defined for exposition only.

[*Note 1*: The instantiation is ill-formed if *range* is not a constant expression ([expr-const]) — *end note*]

(5.3) — Otherwise, S is a destructuring expansion statement and S is equivalent to:

```

{
  init-statement
  constexpropt auto&& [u0, u1, ..., uN-1] = expansion-initializer ;
  S0

```

```

...
SN-1
}

```

where N is the structured binding size of the type of the *expansion-initializer* and S_i is

```

{
  for-range-declaration = ui ;
  compound-statement
}

```

The keyword `constexpr` is present in the declaration of u_0 , u_1 , ..., u_{N-1} if and only if `constexpr` is one of the *decl-specifiers* of the *decl-specifier-seq* of the *for-range-declaration*.

6 [Example 1:

```

constexpr int f(auto const&... Containers) {
  int result = 0;
  template for (auto const& c : {Containers...}) { // OK, enumerating
    expansion statement
    result += c[0];
  }
  return result;
}
constexpr int c1[] = {1, 2, 3};
constexpr int c2[] = {4, 3, 2, 1};
static_assert(f(c1, c2) == 5);

```

— end example]

7 [Example 2:

```

constexpr int f() {
  constexpr std::array<int, 3> arr {1, 2, 3};
  int result = 0;
  template for (constexpr int s : arr) { // OK, iterating expansion
    statement
    result += sizeof(char[s]);
  }
  return result;
}
static_assert(f() == 6);

```

— end example]

8 [Example 3:

```

struct S { int i; short s; };
constexpr long f(S s) {
  long result = 0;
  template for (auto x : s) { // OK, destructuring expansion statement
    result += sizeof(x);
  }
}

```

```

    }
    return result;
}
static_assert(f(S{}) == sizeof(int) + sizeof(short));
— end example ]

```

Modify 8.7.2 [stmt.break]/1 to allow `break` in expansion statements:

- 1 A `break` statement shall be enclosed by ([stmt.pre]) an *iteration-statement* ([stmt.iter]), an *expansion-statement* ([stmt.expand]), or a `switch` statement ([stmt.switch]). The `break` statement causes termination of the ~~smallest~~ *innermost* such enclosing statement; control passes to the statement following the terminated statement, if any.

[Editor's note: We recommend the phrase “continuation portion” in lieu of “loop-continuation portion” to emphasize that an expansion statement is not a loop.]

Modify 8.7.3 [stmt.cont]/1 to allow `continue` in expansion statements:

- 1 A `continue` statement shall be enclosed by ([stmt.pre]) an *iteration-statement* ([stmt.iter]) or an *expansion-statement*. If the innermost enclosing such statement *X* is an *iteration-statement* ([stmt.iter]), the `continue` statement causes control to pass to the ~~loop-continuation portion of the smallest such enclosing statement, that is, to the end of the loop.~~ More precisely, in each of the statements *end of the statement or compound-statement* of *X*. Otherwise, control passes to the end of the *compound-statement* of the current *S_i* ([stmt.expand]).

```

while (foo) {
    {
        // ...
    }
    contin:;
}

```

```

do {
    {
        // ...
    }
    contin:;
} while (foo);

```

```

for (;;) {
    {
        // ...
    }
    contin:;
}

```

~~a `continue` not contained in an enclosing iteration statement is equivalent to `goto contin:`~~

Make a drive-by fix to paragraph 6 of 9.7 [decl.struct.bind] to handle arrays of unknown bound:

- 6 *E* shall not be an array type of unknown bound. If *E* is ~~an~~ *any other* array type with element *T*, the structured binding size of *E* is equal to the number of elements of *E*. Each *SB_i*

is the name of an lvalue that refers to the element i of the array and whose type is T ; the referenced type is T .

Update the list of templated entities in 13.1 [\[temp.pre\]](#)/8:

- 8 An entity is *templated* if it is
- (8.1) — a template,
 - (8.2) — an entity defined ([basic.def]) or created ([class.temporary]) within the *compound-statement* of an expansion statement ([stmt.expand]),
 - (8.3) — an entity defined (~~[basic.def]~~) or created (~~[class.temporary]~~) in a templated entity,
 - (8.4) — a member of a templated entity,
 - (8.5) — an enumerator for an enumeration that is a templated entity, or
 - (8.6) — the closure type of a lambda-expression ([expr.prim.lambda.closure]) appearing in the declaration of a templated entity.

Add to 13.7.1 [\[temp.decls.general\]](#)/3:

- 3 [...] For the purpose of instantiation, the substatements of a constexpr if statement are considered definitions. For the purpose of name lookup and instantiation, the *compound-statement of the expansion-statement* is considered a template definition.

Update 13.8.1 [\[temp.res.general\]](#)/6.1 to permit early checking of expansion statements in dependent contexts.

- 6 The validity of a templated entity may be checked prior to any instantiation.
- [*Note 3*: Knowing which names are type names allows the syntax of every template to be checked in this way. — *end note*]
- The program is ill-formed, no diagnostic required, if
- (6.1) — no valid specialization, ignoring *static_assert-declarations* that fail ([dcl.pre]), can be generated for a templated entity or a substatement of a constexpr if statement ([stmt.if]) within a templated entity and the innermost enclosing template is not instantiated, or
 - (6.x) — no valid specialization, ignoring *static_assert-declarations* that fail, can be generated for the *compound-statement* of an expansion statement and there is no instantiation of it, or
 - (6.2) — [...]

Add the following case to 13.8.3.3 [\[temp.dep.expr\]/3](#) (and renumber accordingly):

- 3 An *id-expression* is type-dependent if it is a *template-id* that is not a concept-id and is dependent; or if its terminal name is
- (3.1) — [...]
 - (3.10) — a *conversion-function-id* that specifies a dependent type, ~~or~~
 - (3.10+) — a name *N* introduced by the *for-range-declaration* *D* of an expansion statement *S* if the type specified for *N* contains a placeholder type and either
 - (3.10+1) — the *expansion-initializer* of *S* is type-dependent or
 - (3.10+2) — *S* is not an iterating expansion statement, or
 - (3.11) — dependent
- or if it names [...]

Add the following case to 13.8.3.4 [\[temp.dep.constexpr\]/2](#) (and renumber accordingly):

- 2 An *id-expression* is value-dependent if
- (2.1) — [...]
 - (2.3) — it is the name of a constant template parameter,
 - (2.3+) — it is a name introduced by the *for-range-declaration* of an expansion statement ([stmt.expand]),
 - (2.4) — [...]

Define the point of instantiation for an expansion statement in a new paragraph at the end of 13.8.4.1 [\[temp.point\]](#):

- 8 For the *compound-statement* of an expansion statement ([stmt.expand]), the point of instantiation is the point of instantiation of its enclosing templated entity, if any. Otherwise, it immediately follows the namespace-scope declaration or definition that contains the expansion statement.

§ 4.1 Feature-test-macro

Add to 15.12 [\[cpp.predefined\]](#):

```
__cpp_expansion_statements 2025XXL
```

§ 5 References

[P0589R0] Andrew Sutton. 2017-02-04. Tuple-based for loops.

<https://wg21.link/p0589r0>

[P1306R0] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2018-10-08. Expansion statements.

<https://wg21.link/p1306r0>

[P1306R1] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2019-01-21. Expansion statements.

<https://wg21.link/p1306r1>

[P1306R2] Dan Katz, Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2024-05-07. Expansion statements.

<https://wg21.link/p1306r2>

[P1306R3] Dan Katz, Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2024-10-14. Expansion statements.

<https://wg21.link/p1306r3>

[P1306R4] Dan Katz, Andrew Sutton, Sam Goodrick, Daveed Vandevoorde, Barry Revzin. 2025-04-24. Expansion Statements.

<https://wg21.link/p1306r4>

[P2996R10] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2025-02-27. Reflection for C++26.

<https://wg21.link/p2996r10>

[P3491R1] Barry Revzin, Wyatt Childers, Peter Dimov, Daveed Vandevoorde. 2025-01-13. `define_static_{string,object,array}`.

<https://wg21.link/p3491r1>